

Описание, инструкция для установки и руководство по использованию программного макета UEBA

1. Описание программного макета

Разработанный программный макет представляет собой **рабочий прототип** системы UEBA на базе нейросетевых методов. Система предназначена для непрерывного мониторинга поведения пользователя на рабочей станции Windows, автоматического обнаружения аномалий в поведенческих паттернах и их классификации.

Макет реализует полный сквозной пайплайн обработки данных:

Сбор телеметрии → Агрегация → Обучение модели → Детекция аномалий → ИБ-классификация → Визуализация в реальном времени

Архитектура состоит из 6 функциональных модулей:

№	Компонент	Назначение	Технология	
1	Сборщик телеметрии	Сбор системных метрик и событий Windows Event Log	Python, psutil, pywin32	
2	Обучающий модуль	Обучение автоэнкодера на нормальном поведении	PyTorch, scikit-learn	
3	Модуль мониторинга	Детекция аномалий в реальном времени	PyTorch, numpy	
4	Классификатор угроз	ИБ-классификация аномалий	Python	
5	Симулятор атак	Генерация тестовых сценариев угроз	Python, socket, ctypes	
6	Веб-дашборд	Визуализация алертов и метрик	Streamlit, Plotly	

2. Структура проект

UEBA/

— data_collection/	# Сбор телеметрии
— collector.py	# Основной сборщик
— collectors/	# Модули сбора (CPU, сеть, Event Log)
— config.yaml	# Конфигурация сбора
— data/aggregated/	# Собранные CSV-файлы
— ml/	# Модуль машинного обучения
— model.py	# Архитектура автоэнкодера
— trainer.py	# Обучение модели
— monitor.py	# Мониторинг и детекция
— threat_classifier.py	# ИБ-классификатор
— calculate_thresholds.py	# Расчёт P95 для классификатора
— artifacts/	# Обученная модель и метаданные
— autoencoder.pth	# Веса модели
— scaler.pkl	# RobustScaler
— metadata.json	# Порог, признаки, P95
— simulations/	# Симуляции атак
— sim_bruteforce.py	# Brute-Force
— sim_cryptomining.py	# Криптоджекинг
— sim_ransomware.py	# Шифровальщик
— sim_portscan.py	# Сканирование портов
— sim_exfiltration.py	# Эксфильтрация
— sim_c2_beacon.py	# C2-маячки
— run_all.py	# Запуск всех симуляций для тестирования
— dashboard/	# Веб-интерфейс
— app.py	# Streamlit-дашборд
— data/	# Рабочие данные
— alerts.json	# Алерты детектора
— venv/	# Виртуальное окружение
— requirements.txt	# Зависимости

3. Требования

- Windows 10/11
- Python 3.10 или выше
- Права администратора

4. Инструкция по использованию

Этап 1. Сбор телеметрии (24 часа)

Запустите сборщик данных. Процесс должен работать не менее 24 часов для накопления достаточной выборки нормального поведения.

1 *python data_collection/collector.py*

В процессе работы в папке `data_collection/data/aggregated/` будут создаваться CSV-файлы с агрегированными признаками.

В период сбора работайте за компьютером в обычном режиме — это обеспечит репрезентативность базового профиля.

Этап 2. Обучение модели (5 мин)

После накопления данных (рекомендуется ≥ 1000 строк) запустите обучение автоэнкодера:

```
1 python ml/trainer.py
```

В результате в папке `ml/artifacts/` будут созданы:

- `autoencoder.pth` — обученная модель
- `scaler.pkl` — скейлер нормализации
- `metadata.json` — порог аномальности, список признаков, значения P95

Этап 3. Мониторинг и детекция аномалий

Запустите модуль мониторинга, при этом будут запрошены права администратора:

```
1 python ml/monitor.py
```

При обнаружении подтверждённой аномалии в консоль выводится информация о типе угрозы, значении MSE, классификация угрозы и рекомендации. Алерты сохраняются в файл `data/alerts.json`.

Этап 4. Тестирование на симуляциях атак (3 часа)

Для проверки работоспособности системы предусмотрены шесть симуляций реальных атак. Запускайте их в отдельном терминале во время работы монитора.

Пример — симуляция Brute-Force атаки:

```
1 python simulations/sim_bruteforce.py
```

Симуляция генерирует события Event ID 4625 (Failed Logon) через системный API Windows. Через 10–20 минут монитор должен зафиксировать аномалию и классифицировать её как **«Подозрительная аутентификация»**.

Аналогично тестируются остальные сценарии:

- `sim_cryptomining.py` — загрузка CPU на 100%
- `sim_ransomware.py` — массовая запись файлов
- `sim_portscan.py` — сканирование портов localhost
- `sim_exfiltration.py` — аномальный исходящий трафик
- `sim_c2_beacon.py` — регулярные C2-соединения

используя

```
2 python simulations/run_all.py
```

можно запустить все сценарии по очереди, каждый будет работать 20 минут, потом будет 20 минут паузы. Весь процесс займет примерно 180 минут.

Этап 5. Визуализация результатов

Запустите веб-дашборд в отдельном терминале:

```
1 streamlit run dashboard/app.py
```

Дашборд откроется в браузере по адресу `http://localhost:8501` и предоставит:

- KPI-метрики: общее число алертов, активность за 24 часа, доминирующий тип угрозы
- Интерактивный график MSE в логарифмическом масштабе
- Круговую диаграмму распределения типов аномалий
- Детальную таблицу алертов с фильтрацией и экспортом в CSV
- Детальный просмотр каждого алерта с рекомендациями

Дашборд поддерживает автоматическое обновление каждые N секунд, что позволяет наблюдать за активностью в режиме реального времени.

3. Итог

Таким образом, разработанный программный макет представляет собой законченный, полностью работоспособный прототип UEBA-системы, готовый к демонстрации, тестированию и дальнейшему развитию.

Основные метрики в ходе тестирования:

- кол-во ложных срабатываний: 0 (теоретически менее чем 0.35 в день)
- полнота срабатываний: 1
- точность классификатора: 0.67

4. Пути развития

- повышение точности классификации аномалий
- увеличение кол-ва отслеживаемых параметров
- анализ не только событий, но и их последовательностей, Переход на LSTM модель
- Создание графического интерфейса для управления всеми этапами сбора данных, обучения, мониторинга
- добавление адаптивности: постепенно собирать новые данные, дообучаться на них, отслеживать средний MSE за день, чтобы меняться вместе с пользователем